Assignment 2: Analyzing and Implementing Divide-and-Conquer Algorithms

Peter Nwochie Tiewah

Student ID: 005064393

Course: MSCS 532 – Algorithms and Data Structures

University of the Cumberlands

October 28, 2026

GitHub repository: https://github.com/Sleekice/mscs-532-dsa

Introduction and Algorithm Design

Divide-and-conquer solves a problem by dividing it into smaller problems, solving each, and combining their solutions. Merge Sort recursively splits an array into halves and merges sorted halves; Quick Sort partitions an array around a pivot and processes each partition. Both sort comparison-based data, but their performance depends differently on input order and partition balance (OpenDSA, n.d.).

Merge Sort: Complexity and Recurrence

For $n > 1$, $T(n) = 2T(n/2) + cn$, with $T(1) = d$. The merging step examines $\Theta(n)$ elements regardless of input order. Thus best-, average-, and worst-case running times are each $\Theta(n \log n)$, giving both $O(n \log n)$ and $\Omega(n \log n)$ bounds.

Substitution (expansion): $T(n) = 2^kT(n/2^k) + kcn$. Setting $k = \log_2 n$ yields $T(n) = nd + cn \log_2 n = \Theta(n \log n)$. For an induction-based upper bound, assume $T(n/2) \leq A(n/2)\log_2(n/2)$ for smaller n. Then $T(n) \leq An \log_2 n - (A - c)n \leq An \log_2 n$ for $A \geq c$; an analogous lower-bound argument establishes $\Theta(n \log n)$.

Recursion tree: There are $\log_2 n$ levels of division, with $\Theta(n)$ total merge work at each internal level plus $\Theta(n)$ at the leaves, giving $\Theta(n \log n)$. Master method: $a = 2$, $b = 2$, $f(n) = \Theta(n) = \Theta(n^{(\log_2 2)})$; Case 2 yields $\Theta(n \log n)$ (OpenDSA, n.d.). The implementation uses $O(n)$ auxiliary list storage and $O(\log n)$ call depth. Its stable merging preserves the order of equal items, useful for sorting records with shared keys.

Quick Sort: Complexity and Recurrence

The program chooses the first item as pivot, uses Lomuto-style partitioning, and processes subranges using an explicit stack rather than Python recursion. For a partition with k items to the left, $T(n) = T(k) + T(n - k - 1) + cn$.

Balanced partitions give approximately $T(n) = 2T(n/2) + cn$. Expansion gives $\Theta(n \log n)$; the recursion tree has $\Theta(\log n)$ levels of $\Theta(n)$ work, and Master Theorem Case 2 confirms $\Theta(n \log n)$. Hence best-case running time is $\Theta(n \log n)$, satisfying $O(n \log n)$ and $\Omega(n \log n)$.

With sorted or reverse-sorted input and a first-item pivot, one partition is empty: $T(n) = T(n - 1) + cn$. Substitution gives $T(n) = T(1) + c\sum_{i=2}^{n} i = \Theta(n^2)$. The recursion tree is a chain of costs proportional to $n, n - 1, \ldots, 2$, totaling $\Theta(n^2)$. The standard Master Theorem is inapplicable because no constant-factor subproblem shrinkage occurs. Worst-case bounds are therefore $O(n^2)$, $\Omega(n^2)$, and $\Theta(n^2)$. For randomly ordered inputs, expected time is $\Theta(n \log n)$; the sorting method is not stable (OpenDSA, n.d.).

Implementation and Experimental Setup

The two Python implementations return sorted copies; each result is checked against sorted(). Benchmarks used Python 3.13.5 on Linux x86_64, with arrays of 500, 1,000, and 2,000 random, sorted, and reverse-sorted integers. Random values use a fixed seed (2026 + n). The reported time is the median of five runs measured using time.perf_counter(); peak traced Python allocations are measured separately using tracemalloc (Python Software Foundation, n.d.-a, n.d.-b). Input construction and reference sorting occur outside the timed section. These allocation measurements are not total process memory, and results will differ across systems.

Results and Interpretation

Input (n=2,000)	Merge (ms)	Quick (ms)	Peak KiB: M / Q
Random	2.8461	2.0233	33.28 / 16.93
Sorted	1.9413	61.5095	39.12 / 140.34
Reverse	1.6607	129.3045	39.12 / 70.78

Table 1. Execution time and traced peak allocations; all 18 measurements are in benchmarks.csv.

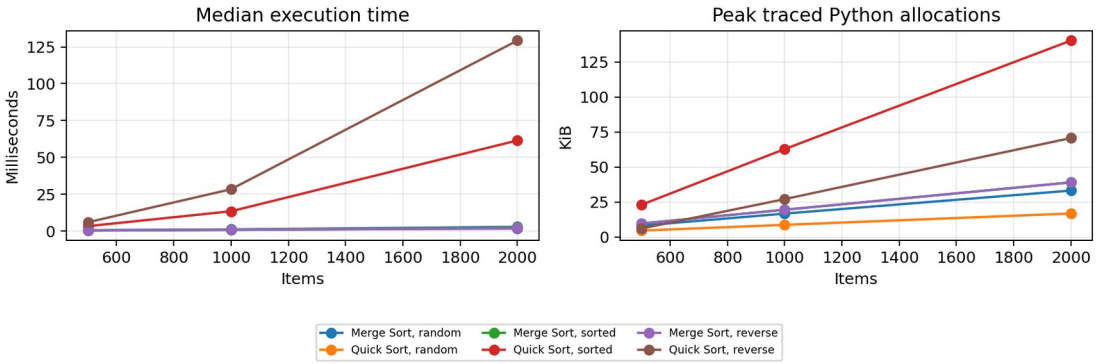


Figure 1. Execution time and peak traced allocations by dataset size and input arrangement.

For 2,000 random values, Quick Sort was faster (2.0233 ms versus 2.8461 ms). On sorted values it took 61.5095 ms versus Merge Sort's 1.9413 ms, and on reverse-sorted values 129.3045 ms versus 1.6607 ms. Sorted Quick Sort became about 4.6 times slower when n doubled from 1,000 to 2,000, consistent with quadratic growth. Merge Sort remained comparatively predictable.

On sorted input, Quick Sort reached 140.34 KiB of traced allocations against Merge Sort's 39.12 KiB; on random input, it used 16.93 KiB against 33.28 KiB. The copied array, Python range tuples, explicit stack, and allocator behavior influence measured memory; these readings should not be equated directly with textbook auxiliary-space bounds.

Conclusion

Merge Sort provides stable, predictable Θ(n log n) performance, while Quick Sort is competitive on random inputs but vulnerable to first-pivot worst cases. Algorithm choice should balance theoretical bounds with measured time, memory usage, input characteristics, and maintainability.

References

OpenDSA. (n.d.). Solving recurrence relations. Virginia Tech. https://opendsa-server.cs.vt.edu/OpenDSA/Books/CS4104/html/Recurrence.html

Python Software Foundation. (n.d.-a). time — Time access and conversions. Python documentation. https://docs.python.org/3/library/time.html

Python Software Foundation. (n.d.-b). tracemalloc — Trace memory allocations. Python documentation. https://docs.python.org/3/library/tracemalloc.html

Code, data, and graph: https://github.com/Sleekice/mscs-532-dsa/tree/master/assignment-02-divide-and-conquer